Design decisions & tradeoffs

Assignment 1

A) Synchronization strategy

(How we keep threads from stepping on each other)

- **What I used:** `synchronized` methods with Java's built-in lock and `wait` / `notifyAll`.

Think of the queue as a **single bathroom** in a house:

- Only **one person** can be inside at a time (one thread in a `synchronized` method).
- Others must **wait outside** until it's free.

Why this is good here:

- Very **simple to write and understand**.
- Fits the assignment goal: *learn* how wait/notify works, not build the fastest library in the world.

Tradeoff (downside):

- Because there is **one lock for everything**, if you have **many producers and consumers**, they all line up for the same bathroom.
- A more advanced tool like `ReentrantLock` can give more control (e.g., `tryLock()`), but it's also more complex.

  🔎 Example:

  - 1 producer + 1 consumer → works great, very little waiting.
  - 10 producers + 10 consumers → still correct, but they might spend more time waiting for the lock than actually doing work.

B) Using `wait()` and `notifyAll()`

What's happening:

- `put()` calls `wait()` when the queue is **full** → "I'll sleep until there's space."
- `take()` calls `wait()` when the queue is **empty** → "I'll sleep until there's an item."
- After adding/removing, we call `notifyAll()` → "Hey everyone, something changed, wake up and check!"

Why **`notifyAll()` and not `notify()`**?

- `notifyAll()` wakes **all threads that are waiting**.
- That way, we avoid tricky bugs where:
    - Only producers get woken up
    - Or only consumers get woken up
      and the system gets stuck.

Tradeoff:

- Some threads wake up, check the condition, and go "oh, I still can't proceed" and go back to sleep.
- This wastes **a tiny bit of CPU**, but in a learning assignment that's okay.

🔎 Example:
Imagine 3 producers and 3 consumers:

- Queue becomes non-empty → we call `notifyAll()`.
- Maybe only **one** consumer can take right now; the others wake up, see nothing for them, and go back to waiting.
  Correct but slightly inefficient. The alternative (`notify()`) is more efficient but easier to get wrong.

C) Poison pill protocol

(How the consumer knows "we are done")

- We use a special number like `POISON_PILL = -1`.
- Producer sends all normal numbers (e.g., `1, 2, 3, 4, 5`) and then sends `-1` at the end.
- When the consumer reads `-1`, it knows:
  "No more real data is coming, I should stop."

Why this is nice:

- Very **easy to understand**.
- No need for extra flags, `stop` booleans, etc.

Tradeoffs:

1. The chosen value (`-1`) must **not** be a valid "real" data value.
    - If your real data might include `-1`, this becomes confusing.
2. With **multiple consumers**, you'd typically need to send **one poison pill per consumer** so each one can stop.

🔎 Simple example with 1 consumer:

- Producer puts: `1, 2, 3, -1`
- Consumer takes:

    - `1` → process
    - `2` → process
    - `3` → process
    - `-1` → stop

🔎 Multi-consumer example (2 consumers):

- You'd usually put *two* `-1` values:
  e.g., `1, 2, 3, -1, -1`
  Each consumer eventually sees a `-1` and stops.

D) Generics (`BoundedBlockingQueue<T>`)

Instead of:

```
class BoundedBlockingQueue {
    // works only with Integer or Object + casts
}
```

we wrote:

```
class BoundedBlockingQueue<T> {
    // works with any type T
}
```

So we can do:

```
BoundedBlockingQueue<Integer> intQueue = new
BoundedBlockingQueue<>(5);
BoundedBlockingQueue<String> stringQueue = new
BoundedBlockingQueue<>(5);
```

Why this is good:

- **Type-safe**: the compiler checks you're using it correctly.
- **Reusable**: same queue class works for `Integer`, `String`, custom objects, etc.

Tradeoff:

- Slightly more confusing if you're new to generics (`<T>` syntax).
- But after you get used to it, it actually makes life easier.

🔎 Example:

- If you had only an Integer queue and later wanted a String queue, you'd need another class or lots of casting.
- With generics: just change the type parameter.

Two improvements if I had more time

1) Better scalability & fairness

Right now:

- Everyone (all producers and consumers) shares **one lock**.
- When something changes, we `notifyAll()` everyone.

This is okay for small examples, but for bigger systems we could:

Improvement idea:

Use `ReentrantLock` + 2 conditions:

- `Condition notEmpty;` → consumers wait on this
- `Condition notFull;` → producers wait on this

Then:

- When we put an item, we `signal()` **only** `notEmpty` (wake a consumer).
- When we take an item, we `signal()` **only** `notFull` (wake a producer).

This:

- Reduces useless wakeups.
- Gives better performance when many threads are running.
- Feels closer to how `ArrayBlockingQueue` in `java.util.concurrent` does it.

🔎 Example scenario:
The queue is full, 5 producers are waiting, 2 consumers are running.

- A consumer takes 1 item → queue not full anymore.
- With `notifyAll()`, all 5 producers wake up.
- With `signal(notFull)`, **only one** producer wakes up.

---

2) Stronger testing & observability

Right now:

- We mostly rely on:
    - Regular unit tests
    - `System.out.printf` to debug

What I'd improve:

1. **More tests**
    - **Interruption test:**
        - Start a producer/consumer.
        - Interrupt the thread.
        - Check that it **stops gracefully** and doesn't just ignore the interrupt.
    - **Stress test:**
        - Many producers + many consumers.
        - Push thousands of items.
        - Check that:
            - All items are processed.
            - No item is lost or processed twice.
    - **Blocking test:**
        - Make the queue full, then call `put()` in another thread and verify it actually **blocks** until space appears.
2. **Better logging**
    - Instead of `System.out`, use a logger (e.g., `java.util.logging` or a small wrapper).
    - Then you can:
        - Turn logging on/off with config.
        - Log timestamps and thread names for debugging concurrency problems.

🔎 Example:
Stress test:

- 3 producers each sending numbers 1–1000 (with IDs like `P1-1`, `P2-5`, etc.).
- 3 consumers processing and storing them in a list.
- At the end:
    - Check the total count = 3000.
    - Check there are no duplicates.
    - Check no item is missing.

This would give a lot more confidence that the queue works even under heavy load.

Assignment 2

1. **Immutable `SalesRecord`**
    - Safer in concurrent / stream operations.
    - Must create a new object if you ever wanted to “change” a record.
2. **CSV parsing by simple `split(",")`**
    - Easy to understand and enough for the assignment.
    - Breaks if fields can contain commas or different formats (real-world CSV is uglier).
3. **Using Streams and `Collectors.groupingBy`**
    - Very concise and matches data-analysis style code.
    - Good for learning modern Java.
    - Slightly harder to grok for beginners than classic loops.
    - Slight overhead for massive files compared to hand-optimized loops.
4. **`Optional` for largest order**
    - Makes it explicit that the result might be empty (e.g., no records).
    - Some people prefer returning `null`, but `Optional` is safer and clearer.

Two improvements if I had more time

1) More robust CSV handling

Right now, parsing assumes:

- No commas inside fields.
- Perfect date/number formats.
- Exactly 8 columns.

Improvements:

- Use a proper CSV library (e.g. OpenCSV / Apache Commons CSV).
- Add better error reporting:
    - Count invalid lines.
    - Log which line failed and why.
- Potentially **skip invalid lines** instead of throwing immediately, depending on requirements.

This would make the tool usable on messy, real-world sales exports.

---

2) Performance and scalability for huge files

Current design:

- Loads **all records into memory** (`List<SalesRecord>`).
- Then runs all analytics on that list.

Improvements:

- Stream the file once and compute some metrics on the fly (e.g. total revenue, summary stats).
- For very large datasets:
    - Consider using **parallel streams** for CPU-heavy calculations.
    - Or write a version that works in chunks rather than reading everything at once.

This would help if the CSV grows from thousands of rows to millions.
